ARQUITETURA DE

SISTEMAS

E-COMMERCE · PARTE 1: USUÁRIOS E CATÁLOGO

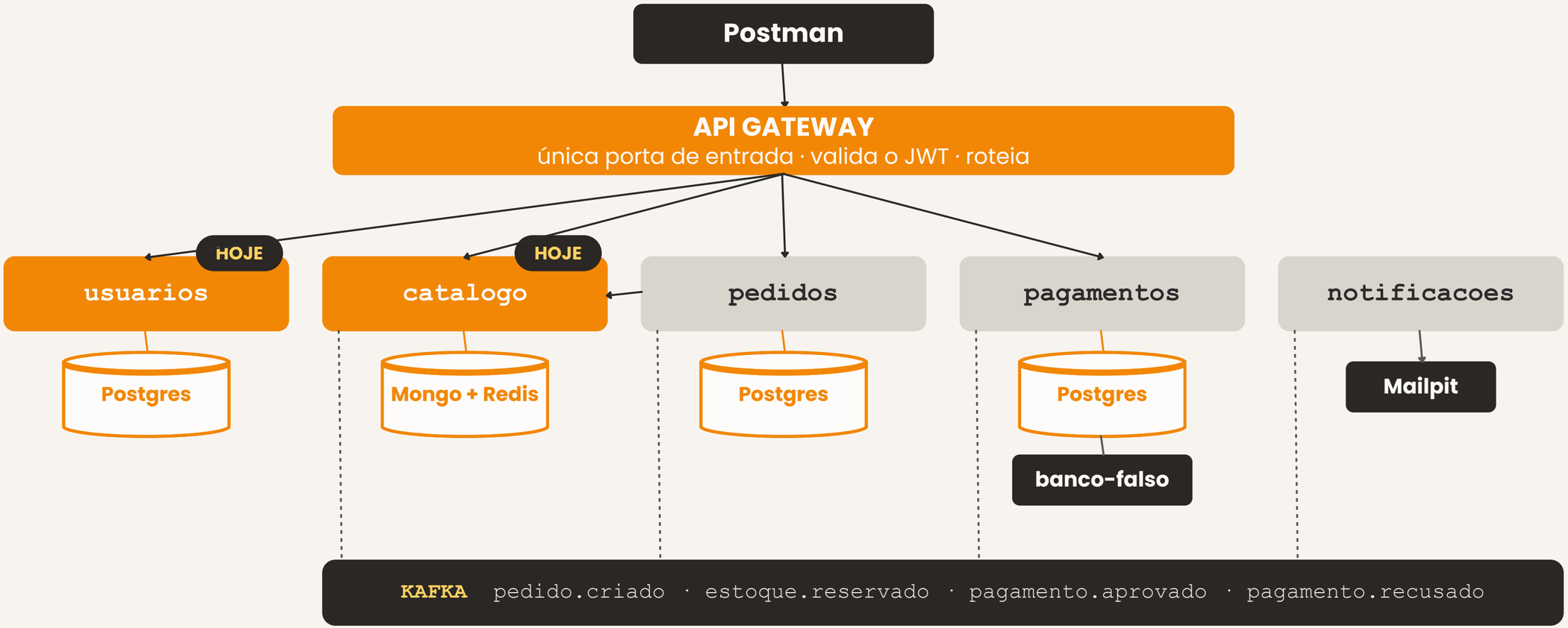
Prof. Augusto Gehrke

HOJE É MÃO NA MASSA

Começa o projeto do G2: um e-commerce em microserviços. Hoje nascem os dois primeiros serviços, cada um com o seu banco, subindo juntos com um comando.

1. **usuarios**: cadastro, login e endereços
2. **catalogo**: produtos, categorias e estoque
3. **Um docker-compose.yml** que sobe os 2 serviços e os 2 bancos
4. **Arquivos .http** testando todas as rotas no VS Code (não teremos front)

O QUE VAMOS CONSTRUIR



Seta cheia: chamada síncrona. O Postman só fala com o gateway; pedidos pergunta ao catálogo o preço na hora da compra.

Tracejado: eventos no Kafka. O checkout vira uma sequência: pedido criado → estoque reservado → pagamento → e-mail.

Cada serviço com o seu banco. O banco-falso simula o gateway de pagamento e falha de propósito.

UMA PEÇA POR AULA

Cada aula acrescenta uma parte ao mesmo projeto, e cada parte resolve um problema que vimos na teoria.

Aula	Entra no projeto	Assunto
Hoje	usuarios + catalogo no Docker Compose	Um banco por serviço, persistência poliglota
2	API Gateway + JWT + pedidos consultando o catálogo	Porta de entrada única, autenticação, chamada síncrona
3	Kafka: pedido criado → estoque reservado; notificacoes	Eventos e comunicação assíncrona
4	pagamentos + banco-falso: compensação e circuit breaker	Transação distribuída e resiliência
5	Redis no catálogo, logs com correlation-id, healthchecks	Cache e observabilidade
6	Deploy no servidor e réplicas do catálogo	Escala e operação

Ao fim de cada aula: o compose sobe e a coleção do Postman passa. É o checkpoint.

ANTES DE CODAR: QUAL BANCO?

Para cada serviço, escolham o banco e defendam a escolha:

usuarios: e-mail que não pode repetir, senha, perfil, vários endereços por pessoa.
catalogo: um celular tem memória e cor; uma camiseta tem tamanho e tecido; um livro tem autor e páginas.

REGRA: cada serviço escolhe o seu. Nada obriga os dois a usarem o mesmo banco.

A ESCOLHA: UM POSTGRES E UM MONGO

Persistência poliglota: cada serviço usa o banco que combina com os seus dados.

USUARIOS → POSTGRES

Estrutura fixa e conhecida: nome, e-mail, senha, perfil.

O e-mail não pode repetir (UNIQUE) e o endereço pertence a um usuário (FK). O banco garante as duas coisas.

CATALOGO → MONGO

Cada categoria tem atributos diferentes. Num banco relacional, viraria uma coluna para cada atributo possível.

No Mongo, cada produto é um documento com os seus próprios campos. E a vitrine é muito mais lida que escrita.

USUARIOS: AS TABELAS (POSTGRES)

```sql
CREATE TABLE usuarios (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nome        TEXT NOT NULL,
  email       TEXT NOT NULL UNIQUE,           -- não repete
  senha_hash TEXT NOT NULL,                   -- bcrypt, nunca a senha
  perfil      TEXT NOT NULL DEFAULT 'cliente' CHECK (perfil IN ('cliente','admin')),
  criado_em   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE enderecos (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id   UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
  apelido      TEXT, cep TEXT NOT NULL, logradouro TEXT NOT NULL, numero TEXT,
  complemento TEXT, cidade TEXT NOT NULL, uf CHAR(2) NOT NULL,
  principal    BOOLEAN NOT NULL DEFAULT false
);
```

SEED OBRIGATÓRIO: **admin@loja.com** / **admin123** (perfil admin) e **cliente@loja.com** / **cliente123** (perfil cliente, com um endereço). Idempotente: rodar duas vezes não duplica.

CATALOGO: AS COLEÇÕES (MONGO)

```
// coleção produtos: um celular
{
  sku: "CEL-001",
  nome: "Smartphone X",
  categoria: "celulares",
  preco: 2499.90,
  ativo: true,
  atributos: {
    memoria: "128GB", cor: "preto"
  },
  estoque: { disponivel: 10, reservado: 0 }
}
```

```
// o mesmo formato, outra categoria
{
  sku: "CAM-010",
  nome: "Camiseta básica",
  categoria: "roupas",
  preco: 59.90,
  ativo: true,
  atributos: {
    tamanho: "M", tecido: "algodão"
  },
  estoque: { disponivel: 40, reservado: 0 }
}
```

categorias: { nome, slug } com **slug** único. **Índices**: **sku** único e **categoria**.

Seed obrigatório: 3 categorias (celulares, roupas, livros) e pelo menos 10 produtos, com atributos diferentes por categoria.

REVISÃO: ROTAS REST

Uma rota = **verbo HTTP** (a ação) + **URL** (o recurso).

```
GET     /produtos         // lista
GET     /produtos/:id     // busca um
POST    /produtos         // cria
PUT     /produtos/:id     // substitui inteiro
PATCH   /produtos/:id     // altera uma parte (ex.: só o estoque)
DELETE  /produtos/:id     // remove
```

✓ **Substantivo no plural:** /produtos, /usuarios. Nada de /criarProduto ou /getUser.

✓ **Filtro vai na query string:** `GET /produtos?categoria=livros&busca=java`

✓ **Sub-recurso no caminho:** `GET /usuarios/me/enderecos`

REVISÃO: STATUS CODES

Código	Significa	No nosso projeto
200 OK	Deu certo e tem conteúdo	GET /produtos
201 Created	Recurso criado	POST /usuarios
204 No Content	Deu certo, sem conteúdo	DELETE /usuarios/me/enderecos/:id
400 Bad Request	Dados inválidos	Cadastro sem e-mail, preço negativo
401 Unauthorized	Não sabemos quem você é	Login errado, token ausente ou vencido
403 Forbidden	Sabemos quem é, mas não pode	Cliente tentando criar produto
404 Not Found	Não existe	GET /produtos/id-que-nao-existe
409 Conflict	Conflita com o que já existe	E-mail já cadastrado, slug repetido
500 Internal Error	Erro nosso	Nunca de propósito

Regra de bolso: 2xx deu certo · 4xx o cliente errou · 5xx o servidor errou.

REVISÃO: REQUISIÇÃO E RESPOSTA HTTP

```
// REQUISIÇÃO
POST /usuarios HTTP/1.1
Host: localhost:3001
Content-Type: application/json
Authorization: Bearer eyJhbGci...

{
  "nome": "Ana",
  "email": "ana@loja.com",
  "senha": "segredo123"
}
```

```
// RESPOSTA
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "7c9e6679-...",
  "nome": "Ana",
  "email": "ana@loja.com",
  "perfil": "cliente"
}     // a senha NUNCA volta
```

✓ **Método e caminho** dizem o que fazer; os **cabeçalhos** dizem como (formato, quem é você).

✓ **O corpo** vai em JSON, e o status code da resposta diz se deu certo antes mesmo de ler o corpo.

REVISÃO: HTTP × HTTPS

HTTP

Tudo viaja em texto puro. Quem estiver no caminho (Wi-Fi do café, provedor) lê a senha do login e copia o token.

Porta 80. Aceitável só dentro de uma rede fechada.

HTTPS = HTTP + TLS

Criptografa a conversa e prova, por certificado, que o servidor é quem diz ser.

Porta 443. Obrigatório em qualquer coisa que tenha login.

NO NOSSO PROJETO: dentro da rede do Docker, os serviços conversam em HTTP. O HTTPS termina na borda: o proxy na frente (no servidor, o Traefik do Coolify, com certificado Let's Encrypt) recebe em HTTPS e repassa. É o mesmo lugar onde, na próxima aula, entra o gateway.

USUARIOS: AS ROTAS

Método	Rota	Acesso	Retornos
POST	/auth/login	público	200 { token, usuario } · 401 e-mail ou senha errados
POST	/usuarios	público	201 usuário criado (sem senha) · 400 dados inválidos · 409 e-mail já existe
GET	/usuarios/me	logado	200 meus dados · 401 sem token
PUT	/usuarios/me	logado	200 dados atualizados · 400 dados inválidos
GET	/usuarios/me/enderecos	logado	200 lista dos meus endereços
POST	/usuarios/me/enderecos	logado	201 endereço criado · 400 dados inválidos
DELETE	/usuarios/me/enderecos/:id	logado	204 removido · 404 não existe ou não é meu
GET	/usuarios	admin	200 lista de usuários · 403 não é admin
GET	/health	público	200 serviço e banco ok

O login devolve um JWT com id, e-mail e perfil, assinado com JWT_SECRET (variável de ambiente). Por enquanto, o próprio serviço valida o token nas rotas /me; na próxima aula, o gateway assume.

CATALOGO: AS ROTAS

Método	Rota	Acesso	Retornos
GET	/produtos?categoria=&busca=	público	200 lista de produtos ativos
GET	/produtos/:id	público	200 produto · 404 não existe
POST	/produtos	admin	201 criado · 400 dados inválidos · 409 SKU já existe
PUT	/produtos/:id	admin	200 atualizado · 400 dados inválidos · 404 não existe
PATCH	/produtos/:id/estoque	admin	200 estoque ajustado · 400 estoque negativo
DELETE	/produtos/:id	admin	204 desativado (ativo = false, não apaga) · 404
GET	/categorias	público	200 lista de categorias
POST	/categorias	admin	201 criada · 409 slug já existe
GET	/health	público	200 serviço e banco ok

Rotas de admin, por enquanto, ficam abertas. O catálogo não deveria saber validar o token de outro serviço; isso é trabalho do gateway, na próxima aula.

UM FORMATO DE ERRO PARA TODOS

Os dois serviços devolvem erros no mesmo formato. Quem consome a API sabe sempre onde olhar.

```
// 409 Conflict em POST /usuarios
{ "erro": "Este e-mail já está cadastrado." }

// 400 Bad Request em POST /produtos
{
  "erro": "Dados inválidos.",
  "detalhes": ["preco deve ser maior que zero", "sku é obrigatório"]
}
```

- ✓ Nunca devolver stack trace nem mensagem do banco para o cliente.
- ✓ Nunca devolver senha_hash, nem dentro da lista de usuários do admin.
- ✓ Validação no serviço, não só no Postman: o banco é a última linha de defesa (UNIQUE, NOT NULL).

ESTRUTURA DO REPOSITÓRIO

```
ecommerce/
├── docker-compose.yml
├── README.md              # como subir + ADRs
├── usuarios/
│   ├── Dockerfile
│   ├── package.json
│   ├── migrations/
│   ├── seeds/
│   └── src/
├── catalogo/
│   ├── Dockerfile
│   ├── package.json
│   ├── seeds/
│   └── src/
└── postman/
    └── ecommerce.postman_collection.json
```

Um repositório, uma pasta por serviço. Os próximos serviços entram como novas pastas.

Cada serviço é independente: seu próprio package.json, Dockerfile e banco. Nenhum importa código do outro.

O compose fica na raiz e sobe tudo.

O README explica como subir e registra as decisões (por exemplo: por que Mongo no catálogo).

TESTANDO A API: ARQUIVOS .HTTP

- ✓ **Requisições em texto,** versionadas no repositório junto com o código. Um arquivo por serviço, na pasta http/.
- ✓ **No VS Code:** extensão REST Client e o botão Send Request.
- ✓ **### separa as requisições;** @ cria variável; # @name guarda a resposta para reaproveitar o token.

```
@usuarios = http://localhost:3001

### Login do cliente
# @name login
POST {{usuarios}}/auth/login
Content-Type: application/json

{ "email": "cliente@loja.com", "senha": "cliente123" }

### Meus dados: reaproveita o token da resposta do login
GET {{usuarios}}/usuarios/me
Authorization: Bearer {{login.response.body.$.token}}
```

Entregável: http/usuarios.http e http/catalogo.http com todas as rotas e os casos de erro (401, 404, 409). Na próxima aula, as URLs viram uma só, a do gateway.

Contato



Augusto Gehrke

https://www.linkedin.com/in/augusto-g-b54a5bb7

augusto.gehrke@amf.edu.br

augustogehrke@outlook.com